MDQ Specification Audit: numeric: mdq-spec

Date: 2026-10-20

Scope: The numeric question type: `docs/question-types/numeric.md` and `schema/numeric.yaml`, with the shared rules it uses in `docs/references/grammar.md`, `docs/references/patterns.md`, `docs/question-types/question-base.md`, `docs/exam.md` and `docs/responses.md`, and the numeric examples of the corpus.

Methodology

`scripts/spec_audit.py` passed (15 steps) before the audit: schemas, docs, Additional Rules tables, lint codes and corpus agree, and mdq-py accepts the corpus.

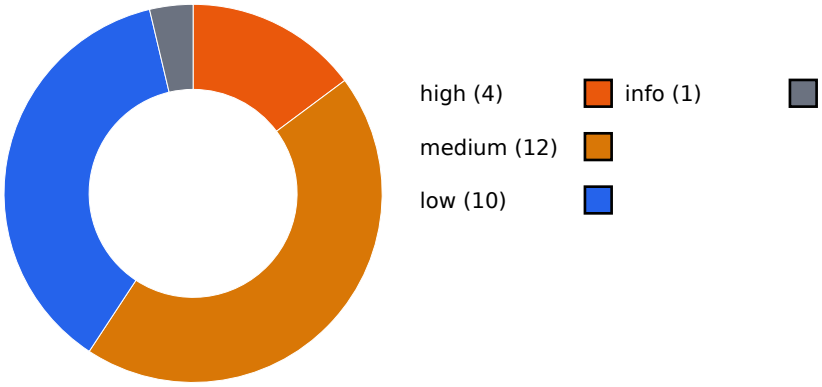
The lead auditor read the shared specs and `docs/question-types/numeric.md`. One subagent turned the grammars in scope into Lark grammars and tested what they accept and reject. A second, independent subagent scanned the numeric specification, its schema, its complete examples (validated with `mdq validate` of mdq-py) and its Additional Rules table.

The lead auditor reviewed every finding of the subagents against the specs before the synthesis.

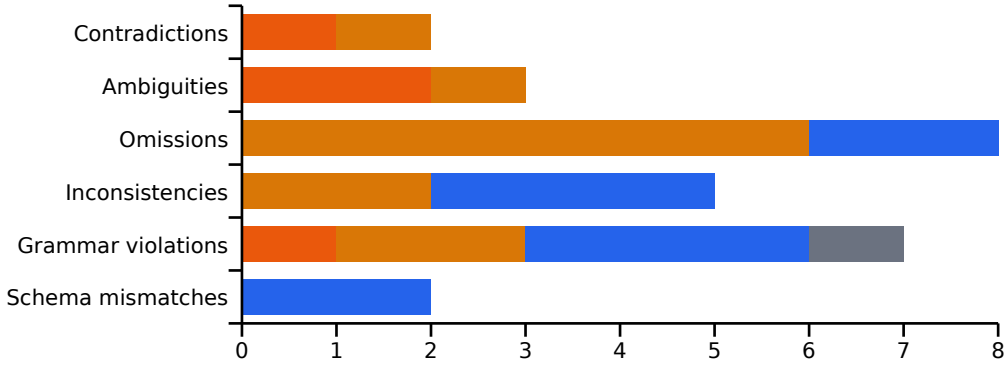
Executive summary

Severity	#
CRITICAL	0
HIGH	4
MEDIUM	12
LOW	10
INFO	1
Total findings	**27**

Findings by severity



Findings by category



Strengths and weaknesses

Strengths

✓ **Grammar violations:** The numeric `answer` grammar accepts all 32 tag lines of the valid numeric corpus and rejects every invalid numeric corpus line except `1/0`. Number literals have no leading zeros, no leading or trailing dot, no exponent, and the sign must touch the value. Rejected (Lark, Earley): `[numeric()]: 60`, `3e5`, `.5`, `007`, `8.`, `26 +- -1`, `[numeric (kg)]: 60`, `60 kg`, `3.14 +- 5% +- 10%`, `[numeric(k[g])]: 60`, `[numeric(k g)]: 60`, U+00A0 after the colon, `- 5`, `00.5`, `1 / 3`, `1.5/2`. Accepted: `-0`, `+26`, `0.50`. (docs/question-types/numeric.md:66-79)

✓ **Grammar violations:** The tolerance grammar allows at most one absolute and one relative tolerance in either order, with optional spaces, and never a fraction or a sign in a tolerance. This agrees with the prose, the domain table and `tolerance.absolute` (a number, minimum 0) in the schema. Accepted: `3.14 +- 5% +- 0.1`, `3.14 +- 0.1 +- 5%`, `[numeric]:3.14+-0.1+-5%`, `42+-1`, `5 +-0%`. Rejected: `5 +- 1 +- 2`, `3.14 +- 5% +- 10%`, `1/3 +- 1/10`, `5 +- 5 %`, `+- 5`, `5 +- 1.5e2`. (docs/question-types/numeric.md:69-76, docs/question-types/numeric.md:122-131, schema/numeric.yaml:76-90)

✓ **Grammar violations:** `UNIT` excludes exactly the 25 code points of `UNICODE_SPACE` and the four brackets, the `unit` pattern of the schema is the same class, and the Unit conversion prose states the same rule. Accepted: `°C`, `km/h`, `R$`, `hab/km²`. Rejected: empty unit, `k g`, `k[g]`, a space before `(`, `(kg)(m)`. (docs/references/grammar.md:89, docs/references/grammar.md:200, schema/numeric.yaml:36, docs/question-types/numeric.md:184-189)

✓ **Grammar violations:** The string pattern of `answer` in the schema matches `sign? value` of the grammar (sign, INTEGER, DECIMAL, fraction), and also rejects a zero denominator as the Answer representation prose requires. (schema/numeric.yaml:29, docs/references/grammar.md:87-88, docs/question-types/numeric.md:71-78, docs/question-types/numeric.md:109-110)

✓ **Grammar violations:** All 12 `numeric_def` lines of the valid fill-in corpus parse with the numeric body grammar, with units and both tolerances, for example `[^litoral/numeric(km)]: 7400 +- 100 +- 2%` and `[^densidade/numeric(hab/km²)]: 23.86 +- 0.5`. (docs/question-types/fill-in.md:158, docs/question-types/fill-in.md:201)

✓ **Inconsistencies:** The UNIT terminal agrees in numeric.md, grammar.md and both schemas: any character except UNICODE_SPACE and the four brackets, with corpus cases for non-ASCII, slash, bracket, space and empty units. (docs/question-types/numeric.md:184-189, docs/references/grammar.md:89, schema/numeric.yaml:36, examples/valid/numeric/unit-non-ascii.mdq.md, examples/invalid/numeric-unit-with-bracket.mdq.md)

✓ **Ambiguities:** Tolerance boundaries are inclusive (<=) and the grading vectors test both edges of absolute and relative tolerances, negative answers and the zero answer. (docs/question-types/numeric.md:151-153, examples/grading/numeric.yaml (Absolute tolerance, Relative tolerance, Negative answer, Zero answer))

✓ **Omissions:** Domain inference is a total table with a rank order, and every row has a corpus example (integer, decimal tolerance, fraction with integer and decimal tolerance, declared wider domain). (docs/question-types/numeric.md:122-138, examples/valid/numeric/fraction-integer-tolerance.mdq.md, examples/valid/numeric/fraction-decimal-tolerance.mdq.md, examples/valid/numeric/domain-decimal-integer-answer.yaml)

✓ **Contradictions:** The rounding mode is fully specified (half away from zero, in decimal arithmetic) with a binary-float trap as example, and the grading vectors test it (2.675 to 2.68). (docs/question-types/numeric.md:162-165, examples/grading/numeric.yaml (Explicit decimal places))

✓ **Schema mismatches:** Schema constraints match the Additional Rules: decimalPlaces and both tolerances have minimum 0, the answer string pattern is the sign? value grammar with a nonzero denominator, and invalid examples cover the negative cases. (schema/numeric.yaml:29, schema/numeric.yaml:47, schema/numeric.yaml:82, schema/numeric.yaml:90, examples/invalid/numeric-negative-absolute-tolerance.yaml, examples/invalid/numeric-negative-decimal-places.yaml)

✓ **Omissions:** The complete Markdown examples of numeric.md and of the numeric blank of fill-in.md validate with no error in mdq-py. (docs/question-types/numeric.md:8-12, docs/question-types/fill-in.md:214-218)

Central risks

• The numeric grading vectors assume exact decimal arithmetic and the shortest decimal form of floats, but the text requires neither, so a conforming float implementation fails the shared vectors on tolerance boundaries.

• Unit handling is optional (MAY) and the outcome of a foreign unit is both an error and a score, so the same response can be graded, scored 0 or rejected depending on the host.

• Rules that depend on how a number is spelled (domain and decimalPlaces inference) have no defined meaning for YAML and JSON documents, which are the canonical AST.

Detailed findings

Contradictions

Severity	Location	Description
HIGH	docs/responses.md:133-142	**F5** A response with a foreign unit is both malformed and scored 0
MEDIUM	docs/question-types/numeric.md:218, docs/question-types/numeric.md:229-230	**F9** missing-tolerance fires on fraction answers, where an exact match is meaningful

F5: A response with a foreign unit is both malformed and scored 0

Location: docs/responses.md:133-142

```
Implementations MAY do unit conversions and rewrite the response to a canonical
form, but they MUST reject a response with a unit that is not compatible with
the question's unit with grade zero.
```

Description: Line 137-139 says a response with an incompatible unit is rejected "with grade zero" (a score). Line 141-142 says "a unit other than the declared one" is a malformed response, which is an error and never a score ("Errors"). Since conversion is MAY, `{value: 1, unit: kg}` for a g question is a score of 1 (after conversion), a score of 0, or a ResponseError, depending on the implementation. A response without a unit for a question that declares one is not covered. How two units are compared (exact string, NFC, case) is not defined.

Impact: The same response gets a score in one implementation and an error in another.

Suggested fix: Decide one rule: (a) a unit other than the declared one is always malformed, and conversion is outside the spec; or (b) define compatible units and the conversion, and malformed only for incompatible ones. State that a bare number is in the declared unit, and how units compare.

F9: missing-tolerance fires on fraction answers, where an exact match is meaningful

Location: docs/question-types/numeric.md:218, docs/question-types/numeric.md:229-230

```
| tolerance          | info     | missing-tolerance              | is not defined for a
`decimal` or `fraction` answer[^5]            |
```

Description: Footnote 5 justifies the rule with "the number of digits the student types decides the outcome". This is true for a decimal answer, but a fraction question compares exact rationals: `1/3` accepts `"1/3"` and `"2/6"` (examples/grading/numeric.yaml, "Fraction"), and no digit count is involved. A tolerance on a fraction question also changes its domain to decimal (table at line 122-128) when written as a decimal.

Impact: Every exact fraction question gets a lint that suggests a change that turns it into a decimal question.

Suggested fix: Restrict `missing-tolerance` to the `decimal` domain, or rewrite the footnote with a reason that holds for fractions.

Ambiguities

Severity	Location	Description
HIGH	docs/question-types/numeric.md: 151-157	**F1** The arithmetic of the tolerance test is not defined
HIGH	docs/question-types/numeric.md: 162-164	**F2** How a float response is read before decimal rounding is not defined
MEDIUM	docs/question-types/numeric.md: 155-157	**F26** The relative tolerance is not defined against the rounded or the unrounded answer

F1: The arithmetic of the tolerance test is not defined

Location: docs/question-types/numeric.md:151-157

```
* Absolute tolerance: The response is accepted if `|answer - response| ≤ abstol`.
```

Description: The spec says that rounding to `decimalPlaces` is done in decimal arithmetic (line 163), but it does not say how `|answer - response|` and `|answer| x relative` are computed. In binary floating point, `3.15 - 3.14` is 0.01000000000000231, so `3.14 +- 0.01` rejects 3.149 (rounded to 3.15). `examples/grading/numeric.yaml`, "Rounding to inferred decimal places", expects score 1 for 3.149: the vectors assume exact (decimal or rational) arithmetic that the text does not require. The same holds for a fraction answer with a relative tolerance.

Impact: Two implementations that both follow the text give different scores on the boundary of every decimal tolerance, which is where authors put their tolerances.

Suggested fix: State that the tolerance test is exact: the answer, the response and both tolerances are read as exact rationals (a decimal as written, a float by its shortest decimal form), and the comparison is done on these rationals.

F2: How a float response is read before decimal rounding is not defined

Location: docs/question-types/numeric.md:162-164

```
many decimal places, half away from zero, in decimal arithmetic (never in
binary floating point: `2.675` rounds to `2.68`). A response with more digits
```

Description: A response is a JSON number or a string (responses.md, "Numeric"). The JSON number 2.675 is the double 2.674999999999999822364316059974953532218933105468 75. Rounded in decimal arithmetic from that exact value it gives 2.67; from its shortest decimal form `2.675` it gives 2.68. `examples/grading/numeric.yaml`, "Explicit decimal places", expects 2.675 (a YAML float) to score 1, so the vectors assume the shortest decimal form. The text does not say it.

Impact: A response sent as a number and the same response sent as a string can score differently, and so can two implementations.

Suggested fix: In responses.md and numeric.md, state that a number response is read as the shortest decimal string that round-trips to the same double, and that this decimal is the value used for rounding and for the tolerance test.

F26: The relative tolerance is not defined against the rounded or the unrounded answer

Location: docs/question-types/numeric.md:155-157

```
If both tolerances are defined, a response is accepted if it passes ANY of those
criteria. When the domain is `decimal`, the response and the answer are first
rounded to [`decimalPlaces`](#decimal-places).
```

Description: The answer is rounded before the tolerance test, but the text does not say if `|answer| × relative` uses the rounded answer, and if the tolerances are rounded too. With `answer: 0.04`, `decimalPlaces: 1` and `relative: 0.5`, the rounded answer is `0.0`: the relative tolerance becomes 0 and only `0.0` passes, while the unrounded answer gives a tolerance of 0.02. `relative-tolerance-around-zero` checks only an answer equal to 0, not one that rounds to 0. No vector in `examples/grading/numeric.yaml` combines a relative tolerance with rounding.

Impact: Implementations can accept different responses for the same question.

Suggested fix: State which value the relative tolerance uses (for example, the answer before rounding), and whether the tolerances are rounded; add a grading vector for it.

Omissions

Severity	Location	Description
MEDIUM	docs/question-types/numeric.md:117-118, docs/question-types/numeric.md:169-173	**F3** decimalPlaces is inferred only from a Markdown body
MEDIUM	docs/responses.md:110-123, docs/responses.md:141-143	**F4** The response forms accepted by each domain are not specified
MEDIUM	docs/question-types/numeric.md:192-202	**F10** Numeric questions have no feedback for responses
MEDIUM	examples/grading/numeric.yaml:1-20	**F11** No grading vector covers units
MEDIUM	docs/question-types/numeric.md:107-108	**F19** Non-finite answers (infinity, NaN) are not excluded
MEDIUM	docs/question-types/fill-in.md:204-209	**F20** A numeric blank in Markdown cannot declare domain or decimalPlaces

Severity	Location	Description
LOW	docs/question-types/numeric.md: 90-91, docs/question-types/numeric.md:100-102	**F23** SHOULD rules of Answer representation are not in Additional Rules
LOW	docs/question-types/numeric.md: 152-153	**F25** The Tolerances section does not map a percentage to tolerance.relative

F3: decimalPlaces is inferred only from a Markdown body

Location: docs/question-types/numeric.md:117-118, docs/question-types/numeric.md:169-173

```
When the frontmatter does not give `decimalPlaces` and the domain is
`decimal`, it is inferred from the body: the larger of the number of decimal
```

Description: A YAML or JSON question has no body, and most YAML/JSON libraries do not keep how a number is spelled. The decimalPlaces rule (line 169-173) does not say what `decimalPlaces` is for `{answer: 3.14, tolerance: {absolute: 0.01}}` or `{answer: 5, domain: decimal}`. `examples/grading/numeric.yaml` infers it from the YAML values ("Rounding to inferred decimal places": 2 places; "Declared decimal domain": 0 places), so a YAML float is counted by its shortest decimal form, and a YAML integer counts 0. The domain inference (line 117-118) has the same gap: "how the value and the absolute tolerance are written" has no meaning for a number in JSON. In Markdown `4.0` is a DECIMAL; in YAML `answer: 4.0` is a whole float, and the text does not say if it infers `integer` or `decimal`. Footnote 3 says that `domain-mismatch` reads values ("`4.0` is whole"), but it does not cover inference. The inferred domain decides if `4.5` is malformed (integer) or graded (decimal).

Impact: An implementation that leaves `decimalPlaces` unset for YAML input, or reads it from a float's full binary expansion, grades differently from the vectors.

Suggested fix: Define both inferences over the answer and the absolute tolerance as values: a string by its digits as written, a number by its shortest decimal form (an integer counts 0 places).

F4: The response forms accepted by each domain are not specified

Location: docs/responses.md:110-123, docs/responses.md:141-143

```
A string carries an exact rational, which no float can represent. It MUST be
used when the question's `domain` is `fraction` and the value is not exactly
representable as a decimal:
```

Description: responses.md gives one example of a malformed value (a decimal for an `integer` domain) but no rule per domain. The vectors fix answers the text does not give: for `domain: fraction`, the decimal responses 0.3333 and "0.5" score 0 and are not malformed; for an `integer` domain, "1/2" is malformed. It is not stated whether a fraction string is valid for a `decimal` question, whether `"4/2"` or `4.0` is valid for an `integer` question (footnote 3 of numeric.md says the lint reads values, so `4.0` is whole), or whether exponents (1e3, `"1e3"`) and a leading + are accepted in a string response. The sentence "The string MUST be a valid decimal" (line 119) also excludes integer strings such as "1822", which the vectors accept.

Impact: Implementations disagree on which responses are malformed and which are scored, which changes the student's grade or the error the host gets.

Suggested fix: Add a table to responses.md, "Numeric": for each domain, which response forms are valid (integer, decimal, fraction, number, string) and which are malformed, with the string grammar (`sign? value` of numeric.md or a stated extension).

F10: Numeric questions have no feedback for responses

Location: docs/question-types/numeric.md:192-202

```
Grading is binary: a response inside the accepted tolerance scores 1, and any
other response scores 0.
```

Description: Short answer has `reject` patterns with feedback and `incorrectFeedback`; choice lists have feedback per choice. A numeric question has no way to attach feedback to an incorrect response, not even a fixed `incorrectFeedback`, and no way to recognize a known wrong answer (for example, a unit error of a factor 1000). The spec does not say that this is intended.

Impact: Authors move numeric questions to short-answer regexes to give feedback, and lose tolerance and domain checks.

Suggested fix: Either state that numeric has no feedback by design, or add `incorrectFeedback` (as in short-answer and the blanks of fill-in) and, optionally, `reject` values with tolerance and feedback.

F11: No grading vector covers units

Location: examples/grading/numeric.yaml:1-20

```
#   examples: { response: number | string, score: 0 | 1 | "malformed" }[]
```

Description: The response type in the header of the vector file does not include the `{value, unit}` form of responses.md, line 125-131, and no vector has a `unit`. The malformed cases for a unit (a unit on a question without one, a unit other than the declared one) are not tested.

Impact: The unit rules of responses.md have no shared test, so implementations can drift.

Suggested fix: After the unit rule is decided, add the `{value, unit}` form to the header and vectors for each case.

F19: Non-finite answers (infinity, NaN) are not excluded

Location: docs/question-types/numeric.md:107-108

```
A document written directly as JSON/YAML/dict, rather than parsed from
Markdown, can choose freely between numeric and string representations.
```

Description: A YAML document can give `answer: .inf` or `answer: .nan`. The spec does not exclude them, and `schema/numeric.yaml:27` (`type: number`) accepts them. The domain table, the tolerance test and the rounding are not defined for these values. `mdq validate --format yaml` on `{type: numeric, stem: x, answer: .inf}` crashes with `OverflowError: cannot convert float infinity to integer`, and `.nan` exits with `error: cannot convert float NaN to integer`.

Impact: Implementations behave differently on such documents: the reference one crashes instead of reporting a diagnostic.

Suggested fix: State that `answer` and the tolerances are finite numbers, and give the error code for a non-finite value.

Exploitability conditions: A YAML or JSON document (YAML has `.inf` and `.nan`; some JSON parsers accept `Infinity` and `NaN`).

F20: A numeric blank in Markdown cannot declare domain or decimalPlaces

Location: docs/question-types/fill-in.md:204-209

```
The `numeric_body` and `unit` non-terminals represent, respectively, a numeric
value with tolerance and an optional unit of measurement. Both are defined in
the [body](numeric.md#body) section of the numeric question type. The `blank`
```

Description: `NumericBlank` in `schema/fill-in.yaml` has `domain` and `decimalPlaces`, and numeric.md sets both in the frontmatter. The fill-in frontmatter table has no field for them, and the `numeric_def` syntax has no place for them. So a Markdown numeric blank always gets the inferred domain and places. The spec does not say that this limit is intended, and no example in `examples/valid/fill-in/` sets them.

Impact: An author cannot write a decimal blank with an integer answer (42 in a `decimal` domain), or set the precision of a blank, in Markdown. The YAML form can express questions that the Markdown form cannot.

Suggested fix: Either state that a Markdown numeric blank always uses the inferred domain and places, or add a frontmatter map (blank id to `domain`/`decimalPlaces`), as fill-in already does for `preAccept` and `incorrectFeedback`.

F23: SHOULD rules of Answer representation are not in Additional Rules

Location: docs/question-types/numeric.md:90-91, docs/question-types/numeric.md:100-102

```
An integer number like `42` or `-1` SHOULD be coerced to an integral value.
It SHOULD be kept as a string if it is not representable in the 32-bit signed
integer range.
```

Description: "Numeric conversions SHOULD NOT be attempted when there is loss of precision" (line 90-91) and the two SHOULD rules of line 100-102 have no row in the Additional Rules table. They are rules for the parser output, not for the document, so no lint code can report them, but the spec does not say this.

Impact: The requirement levels in the text and in the table do not agree.

Suggested fix: State that the rules of "Answer representation" apply to the parser output and have no diagnostic, or reword them without RFC 2119 keywords.

F25: The Tolerances section does not map a percentage to tolerance.relative

Location: docs/question-types/numeric.md:152-153

```
where tol is calculated as `|answer| × percentage_number ÷ 100`.
```

Description: The formula uses the percentage as written in Markdown. The YAML/JSON form stores `tolerance.relative` as a fraction (`0.05` for 5%), but numeric.md says this only in footnote 4 of the Additional Rules ("`0.05` is 5%") and footnote 2 uses `|answer| × relative`. The section does not give the formula for the stored field.

Impact: An author who writes YAML can put 5 for 5%, which accepts a response 500% off (only an info diagnostic reports it).

Suggested fix: State that `+- p%` is stored as `tolerance.relative: p / 100` and give the formula as `|answer - response| ≤ |answer| × relative`.

Inconsistencies

Severity	Location	Description
MEDIUM	docs/question-types/numeric.md:210, docs/question-types/numeric.md:214	**F8** A non-whole answer of an integer question gets two diagnostics
MEDIUM	docs/question-types/numeric.md:111-113	**F21** A non-whole answer under domain fraction is a SHOULD, but only an info diagnostic

Severity	Location	Description
LOW	docs/responses.md:118-119	**F6** Unclear sentence about decimal string responses
LOW	docs/question-types/numeric.md:211-218	**F22** Additional Rules has rows that the prose does not state
LOW	docs/question-types/numeric.md:5-6, docs/question-types/numeric.md:36-37	**F24** The intro says that a numeric question always declares a tolerance

F8: A non-whole answer of an integer question gets two diagnostics

Location: docs/question-types/numeric.md:210, docs/question-types/numeric.md:214

```
| answer             | warning  | answer-outside-domain          | must be a whole number when
`domain` is "integer"[^6]              |
```

Description: `answer-outside-domain` (warning) fires when `domain` is `integer` and the answer is not whole. `domain-mismatch` (info, footnote 3) fires when the declared domain is narrower than the values, and footnote 3 gives "`integer` ... with a number that is not whole" as a case. So `domain: integer` with `answer: 3.5` matches both rows. Also, the rule text says "must", which is an error level, for a warning.

Impact: Duplicate diagnostics, or implementations that report only one of them; corpus lint files cannot tell which is right.

Suggested fix: Decide which code covers the answer and which covers the tolerance (for example, `answer-outside-domain` for the answer, `domain-mismatch` only for a decimal absolute tolerance under `integer`/`fraction`), and use "should" for a warning.

F21: A non-whole answer under domain fraction is a SHOULD, but only an info diagnostic

Location: docs/question-types/numeric.md:111-113

```
When `domain` is `fraction`, such a document SHOULD give `answer` as a
fraction string (`"3/4"`) or as an integer (`2` or `"2"`). A number that is
not whole, such as `0.75`, is a decimal and contradicts the domain.
```

Description: The SHOULD maps to a warning by the levels of the Additional Rules (SHOULD: warning). The only row that covers this case is `domain-mismatch` (info, footnote 3). For `domain: integer`, the same defect (a non-whole answer) gets the warning `answer-outside-domain`. `examples/valid/numeric/fraction-answer-float.lint.json` confirms that `answer: 0.25` with `domain: fraction` gets only `domain-mismatch` at info level.

Impact: Two cases with the same text ("contradicts the domain") get different severities, and the SHOULD has no warning.

Suggested fix: Decide one level for a non-whole answer under a narrower domain: extend `answer-outside-domain` to `fraction`, or change the SHOULD to MAY.

F6: Unclear sentence about decimal string responses

Location: docs/responses.md:118-119

```
The response of a decimal value may also be represented as a string, which
MAY signal the system to use decimal arithmetic instead of floating point. The string MUST be a
 valid decimal.
```

Description: "which systems MAY flag the system to use decimal arithmetic" is not a correct sentence, and it suggests that the arithmetic depends on whether the response is a string. If the comparison is exact (see the tolerance arithmetic finding), the string form only keeps digits that a float would lose.

Impact: Readers can think that a number response is graded in floating point.

Suggested fix: Rewrite: a decimal response can also be a string; it is read as the exact decimal it spells.

F22: Additional Rules has rows that the prose does not state

Location: docs/question-types/numeric.md:211-218

```
| tolerance.absolute | error    | schema-error                  | must not be negative
(schema)                                        |
```

Description: The prose of numeric.md does not state these rows: `tolerance.absolute` must not be negative; `relative-tolerance-around-zero` (only footnote 2 explains it); `relative-tolerance-over-one` (only footnote 4); `missing-tolerance` (only footnote 5). The "Tolerances" section (line 146-157) does not say that a tolerance is non-negative, and the grammar (no sign on `abstol`) is the only hint for Markdown.

Impact: A reader of the prose does not learn these rules; the table is the only source.

Suggested fix: Add the rules to the "Tolerances" section: both tolerances are non-negative, a relative tolerance is a fraction (`5%` is `0.05`), and a relative tolerance around a zero answer and a missing tolerance on a decimal answer are flagged.

F24: The intro says that a numeric question always declares a tolerance

Location: docs/question-types/numeric.md:5-6, docs/question-types/numeric.md:36-37

Mechanical fix

```
Numeric questions expect a single numeric answer. They declare both the correct
answer and tolerance.
```

Description: Line 5-6 and line 36-37 ("followed by the expected numeric value and tolerance") read as if the tolerance is required. The grammar (`tolerances?`), the example right below (`[numeric(g)]: 1000`) and `missing-tolerance` make it optional.

Impact: Minor: a reader can think that `[numeric]: 42` is invalid.

Suggested fix: Write "the correct answer and, optionally, a tolerance" in both places.

Grammar violations

Severity	Location	Description
HIGH	docs/question-types/numeric.md:78, docs/question-types/numeric.md:109-110	**F12** The fraction rule accepts a zero denominator that the corpus rejects as parse-error
MEDIUM	docs/question-types/fill-in.md:158, docs/question-types/fill-in.md:201	**F13** numeric_def has no leading ws?, unlike the numeric answer and the other blank definitions
MEDIUM	docs/references/grammar.md:59-62	**F14** ws? at the start of a block has no limit of three spaces

Severity	Location	Description
LOW	docs/question-types/fill-in.md:158, docs/question-types/fill-in.md:201	**F15** numeric_def gives two ws? in a row, which makes the grammar ambiguous
LOW	docs/question-types/fill-in.md:158, docs/question-types/fill-in.md:201	**F16** The two numeric_def snippets of fill-in.md are written differently
LOW	docs/references/grammar.md:89	**F17** UNIT does not say if a markdown escape or code span in the unit is source text
INFO	docs/question-types/numeric.md:68-74	**F18** Common number spellings are a bare parse-error with no lint to explain them

F12: The fraction rule accepts a zero denominator that the corpus rejects as parse-error

Location: docs/question-types/numeric.md:78, docs/question-types/numeric.md:109-110

Mechanical fix

```
fraction     : INTEGER "/" INTEGER
```

Description: INTEGER matches 0, so the grammar accepts `[numeric]: 1/0` (tested with Lark, Earley parser). The corpus case `examples/invalid/numeric-zero-denominator.mdq.md` expects `parse-error` for that input, and the schema pattern of `answer` in `schema/numeric.yaml` uses `[1-9][0-9]*` for the denominator. The only prose rule about a zero denominator (numeric.md:109-110) applies to documents written directly as JSON/YAML, and the Additional Rules table maps it to `malformed-numeric-answer`, which `examples/invalid/numeric-malformed-fraction-answer.lint.json` expects for `answer: "1/0"`. So the same defect has two codes, and only one is stated in the spec.

Impact: An implementation that follows the grammar accepts `1/0` and then reports `malformed-numeric-answer` or `schema-error`, so it fails the corpus case. The spec gives three different outcomes for one input.

Suggested fix: Use a terminal with no zero for the denominator, for example `fraction : INTEGER "/" DENOMINATOR` with `DENOMINATOR : /[1-9][0-9]*/`, as the schema pattern and the corpus already do.

Exploitability conditions: Markdown body with a fraction whose denominator is 0.

F13: numeric_def has no leading ws?, unlike the numeric answer and the other blank definitions

Location: docs/question-types/fill-in.md:158, docs/question-types/fill-in.md:201

```
numeric_def      : "[^" SLUG "/numeric" unit? "]" ws? ":" ws? numeric_body
```

Description: `answer` of numeric (numeric.md:67) and `accept_def` and `reject_def` of fill-in (fill-in.md:159-160) start with `ws?`. `numeric_def` at line 158 and line 201 does not. With Lark, `[^a/numeric]: 5` (three spaces of indentation, a valid CommonMark paragraph start) is rejected by `numeric_def`, but `[numeric]: 5` is accepted by `answer` and `[^a/short-answer]: x` by `accept_def`.

Impact: An indented numeric blank definition is a parse error by the grammar while the same indentation is valid on the other definitions. Implementations can disagree on which reading to follow.

Suggested fix: Add `ws?` at the start of `numeric_def` in both snippets, or remove it from all definitions and state the indentation rule once.

Exploitability conditions: A numeric blank definition with 1 to 3 spaces of indentation.

F14: ws? at the start of a block has no limit of three spaces

Location: docs/references/grammar.md:59-62

```
When a rule starts with `ws?` at the start of a block, implementations MUST
read the source text and accept only spaces and tabs there.
```

Description: grammar.md:66-72 limits the leading ws? to three spaces only for the content of a list item and for feedback and comment markers. For a rule at the start of a block, such as answer of numeric (ws? "[" "numeric" unit? "]" ...), the text gives no limit, and ws is /[\t]+/. The grammar accepts [numeric]: 42 (four spaces) and \t[numeric]: 42. CommonMark reads both lines as an indented code block, not as a paragraph.

Impact: An implementation that applies the grammar to the source text accepts an indented code block as the answer. An implementation that uses the CommonMark block structure rejects it. The same document gives a question or a parse error.

Suggested fix: State in grammar.md:59-62 that ws? at the start of a block stands for at most three spaces, with tabs expanded as in grammar.md:70-71, the same as for a list item.

Exploitability conditions: A tag line with four or more columns of indentation.

F15: numeric_def gives two ws? in a row, which makes the grammar ambiguous

Location: docs/question-types/fill-in.md:158, docs/question-types/fill-in.md:201

Mechanical fix

```
numeric_def : "[^" SLUG "/" "numeric" unit? "]" ws? ":" ws? numeric_body
```

Description: numeric_body (numeric.md:68) starts with ws?, and numeric_def puts another ws? before it. For every corpus line, such as [^minima/numeric(°C)]: -14 +- 0.5, Lark (Earley, ambiguity='explicit') returns an _ambig node: the space after : is either the ws of numeric_def or the ws of numeric_body. answer of numeric (":" numeric_body) does not have this problem.

Impact: No effect on the value, but an LALR parser built from the snippet has a conflict, and a reader cannot tell which rule owns the space.

Suggested fix: Remove the ws? after ":" in numeric_def, as answer does.

F16: The two numeric_def snippets of fill-in.md are written differently

Location: docs/question-types/fill-in.md:158, docs/question-types/fill-in.md:201

Mechanical fix

```
numeric_def : "[^" SLUG "/" "numeric" unit? "]" ws? ":" ws? numeric_body
```

Description: Line 158 uses the literal "/numeric" and line 201 uses "/" "numeric". Without %ignore the two accept the same strings (tested: [^a/numeric(km)]: 5 passes both), but the reader must check this. The same rule is defined twice with different text.

Impact: A later edit to one copy can make the two definitions differ.

Suggested fix: Use the same text in both snippets, or keep the rule in one place and refer to it.

F17: UNIT does not say if a markdown escape or code span in the unit is source text

Location: docs/references/grammar.md:89

```
UNIT        : /[^()\[\]\t\n\v\f\r \x85\xa0\u1680\u2000-\u200a\u2028\u2029\u202f\u205f\u3000]+/
```

Description: `UNIT` accepts the backslash and the backtick. The grammar accepts `[numeric(m\*s)]: 42` and `` `[numeric(m)]: 42. Markdown reads \* as an escaped * and m as a code span. The spec says that the source text is read only for the leading ws? (grammar.md:59-62), so the unit can be m\*s or m*s` ``.

Impact: Two implementations can store different `unit` strings for the same tag.

Suggested fix: State that the tag line, including the unit, is read as source text with no markdown inline processing, or exclude `\` and the backtick from `UNIT`.

Exploitability conditions: A unit with a backslash or a backtick.

F18: Common number spellings are a bare parse-error with no lint to explain them

Location: docs/question-types/numeric.md:68-74

```
numeric_body : ws? sign? value tolerances? ws?
```

Description: The grammar rejects `[numeric]: −5` (U+2212 MINUS SIGN), `[numeric]: 3.14 ± 0.1`, `[numeric]: 3,14` (decimal comma, the usual form in pt-BR), `[numeric]: 1 000` and `[numeric]: 6.022e23`. Only the exponent has a corpus case (`numeric-exponent`). The spec does not say that these forms are intentionally rejected, and there is no lint code that tells the author what to write.

Impact: Authors who write Brazilian or typographic number forms get a generic parse-error.

Suggested fix: List the rejected forms in the Body section, and consider a lint code with a hint for U+2212, ± and the decimal comma.

Schema mismatches

Severity	Location	Description
LOW	schema/numeric.yaml:48-50, schema/numeric.yaml:86-89	**F7** Schema descriptions of decimalPlaces and relative disagree with the spec
LOW	schema/numeric.yaml:43	**F29** Schema description of domain omits inference and its effect

F7: Schema descriptions of decimalPlaces and relative disagree with the spec

Location: schema/numeric.yaml:48-50, schema/numeric.yaml:86-89

Mechanical fix

```
The number of decimal places used in the decimal value representation.
```

Description: numeric.md defines `decimalPlaces` as the precision of the comparison (both values are rounded to it), not of a representation. The `relative` description gives `|submitted - answer| / |answer| <= relative`, which divides by zero for a zero answer; numeric.md (line 152-153 and footnote 2) uses `|answer - response| <= |answer| x relative`, which is defined for 0.

Impact: A reader of the schema alone implements a different rule.

Suggested fix: Copy the definitions of numeric.md: "the precision of the comparison: the response and the answer are rounded to this many places" and `|submitted - answer| <= |answer| * relative`.

F29: Schema description of domain omits inference and its effect

Location: schema/numeric.yaml:43

Mechanical fix

```
description: The domain/kind of number for the response.
```

Description: numeric.md ("Number type/domain") says that `domain` is inferred from the answer and the absolute tolerance when absent, that the parsed document carries the inferred value, and that the domain decides which responses are malformed and whether `decimalPlaces` applies. The schema description says none of this. The same text is in `schema/fill-in.yaml` (`NumericBlank.domain`).

Impact: A reader of the schema alone cannot know the default of `domain` or what it changes.

Suggested fix: Describe `domain` as in numeric.md: inferred when absent (integer < fraction < decimal, the widest of the answer and the absolute tolerance), and it decides which responses are valid.

Prioritized recommendations

P1: Define numeric values and the tolerance test in exact rational arithmetic, with floats read by their shortest decimal form. (Findings: F1, F2, F3, F26)

P1: Decide one rule for responses with units and test it with vectors. (Findings: F5, F11, F17)

P1: Add a domain x response form table and finite-number rules to responses.md. (Findings: F4, F6, F19)

P2: Give each numeric domain and tolerance defect one diagnostic with the right level, and state each Additional Rules row in the prose. (Findings: F8, F9, F21, F22, F23, F25)

P2: Align numeric blanks with numeric questions and limit block indentation as CommonMark does. (Findings: F13, F20, F14)

P2: Apply the mechanical fixes in one pass: zero denominator in the grammar, numeric_def text, intro wording, schema descriptions. (Findings: F7, F12, F15, F16, F24, F29)

P3: Decide on numeric feedback and on hints for rejected number spellings. (Findings: F10, F18)

GitHub issues

Each block below is a complete issue, ready to paste.

````
--- ISSUE 1 ---
# [Spec] Define numeric values and the tolerance test in exact arithmetic

**Labels:** high, medium

## Problem

The tolerance test, the reading of float responses, the inference of domain and decimalPlaces
from YAML values, and the base of the relative tolerance are not defined. Options: (a)
recommended: every number (answer, response, tolerances) is an exact rational; a float is read
by its shortest round-trip decimal form; inference reads that form (a whole number with no
decimals is integer); the relative tolerance uses the answer before rounding; tolerances are
not rounded. (b) Allow floating point and add an epsilon to the comparison; simpler to
implement but the vectors and the 2.675 example must change.

## Evidence

- `docs/question-types/numeric.md:151-157`: The arithmetic of the tolerance test is not defined

  ```markdown
  * Absolute tolerance: The response is accepted if `|answer - response| ≤ abstol`.
  ```
- `docs/question-types/numeric.md:162-164`: How a float response is read before decimal
rounding is not defined

  ```markdown
  many decimal places, half away from zero, in decimal arithmetic (never in
  binary floating point: `2.675` rounds to `2.68`). A response with more digits
  ```
- `docs/question-types/numeric.md:117-118, docs/question-types/numeric.md:169-173`:
decimalPlaces is inferred only from a Markdown body

  ```markdown
  When the frontmatter does not give `decimalPlaces` and the domain is
  `decimal`, it is inferred from the body: the larger of the number of decimal
  ```
- `docs/question-types/numeric.md:155-157`: The relative tolerance is not defined against the
rounded or the unrounded answer

  ```markdown
  If both tolerances are defined, a response is accepted if it passes ANY of those
  criteria. When the domain is `decimal`, the response and the answer are first
  rounded to [`decimalPlaces`](#decimal-places).
  ```

## Impact

- **The arithmetic of the tolerance test is not defined:** Two implementations that both follow
 the text give different scores on the boundary of every decimal tolerance, which is where
authors put their tolerances.
- **How a float response is read before decimal rounding is not defined:** A response sent as a
 number and the same response sent as a string can score differently, and so can two
implementations.
- **decimalPlaces is inferred only from a Markdown body:** An implementation that leaves
`decimalPlaces` unset for YAML input, or reads it from a float's full binary expansion, grades
differently from the vectors.
- **The relative tolerance is not defined against the rounded or the unrounded answer:**
Implementations can accept different responses for the same question.

## Suggested fix

- **The arithmetic of the tolerance test is not defined:** State that the tolerance test is
exact: the answer, the response and both tolerances are read as exact rationals (a decimal as
written, a float by its shortest decimal form), and the comparison is done on these rationals.
- **How a float response is read before decimal rounding is not defined:** In responses.md and
numeric.md, state that a number response is read as the shortest decimal string that round-
trips to the same double, and that this decimal is the value used for rounding and for the
tolerance test.
- **decimalPlaces is inferred only from a Markdown body:** Define both inferences over the
answer and the absolute tolerance as values: a string by its digits as written, a number by its
````

```
 shortest decimal form (an integer counts 0 places).
- **The relative tolerance is not defined against the rounded or the unrounded answer:** State
which value the relative tolerance uses (for example, the answer before rounding), and whether
the tolerances are rounded; add a grading vector for it.

## Acceptance criteria

- [ ] numeric.md states the arithmetic of rounding and of the tolerance test
- [ ] responses.md states how a number response is read
- [ ] Inference of domain and decimalPlaces is defined for YAML/JSON values
- [ ] Grading vectors cover a relative tolerance with rounding and a float on a tolerance
boundary

--- END ISSUE 1 ---
```

````
--- ISSUE 2 ---
# [Spec] Specify the valid answer and response values for each domain

**Labels:** medium, low

## Problem

responses.md gives examples but no rule for which response forms each domain accepts, and the
answer can be infinite or NaN. Recommended: a table in responses.md (domain x form: integer,
decimal, fraction, number, string) that states valid or malformed, the string grammar of a
response, and that answer, response and tolerances are finite.

## Evidence

- `docs/responses.md:110-123, docs/responses.md:141-143`: The response forms accepted by each
domain are not specified

  ```markdown
  A string carries an exact rational, which no float can represent. It MUST be
  used when the question's `domain` is `fraction` and the value is not exactly
  representable as a decimal:
  ```
- `docs/responses.md:118-119`: Unclear sentence about decimal string responses

  ```markdown
  The response of a decimal value may also be represented as a string, which
  MAY signal the system to use decimal arithmetic instead of floating point. The string MUST be
 a valid decimal.
  ```
- `docs/question-types/numeric.md:107-108`: Non-finite answers (infinity, NaN) are not excluded

  ```markdown
  A document written directly as JSON/YAML/dict, rather than parsed from
  Markdown, can choose freely between numeric and string representations.
  ```

## Impact

- **The response forms accepted by each domain are not specified:** Implementations disagree on
 which responses are malformed and which are scored, which changes the student's grade or the
error the host gets.
- **Unclear sentence about decimal string responses:** Readers can think that a number response
 is graded in floating point.
- **Non-finite answers (infinity, NaN) are not excluded:** Implementations behave differently
on such documents: the reference one crashes instead of reporting a diagnostic.

## Suggested fix

- **The response forms accepted by each domain are not specified:** Add a table to
responses.md, "Numeric": for each domain, which response forms are valid (integer, decimal,
fraction, number, string) and which are malformed, with the string grammar (`sign? value` of
numeric.md or a stated extension).
- **Unclear sentence about decimal string responses:** Rewrite: a decimal response can also be
a string; it is read as the exact decimal it spells.
- **Non-finite answers (infinity, NaN) are not excluded:** State that `answer` and the
tolerances are finite numbers, and give the error code for a non-finite value.

## Acceptance criteria

- [ ] responses.md has the table and the string grammar
- [ ] Non-finite answers have an error code and a corpus case
- [ ] Vectors cover each malformed cell

--- END ISSUE 2 ---
````

````
--- ISSUE 3 ---
# [Spec] Decide how numeric responses with units are graded

**Labels:** high, medium, low

## Problem

A foreign unit is both malformed and scored 0, conversion is MAY, a bare number for a question
with a unit is not covered, and the unit string of a tag is not defined under markdown escapes.
 Options: (a) recommended: the unit of a response must equal the declared unit (exact string
after NFC) or be absent (a bare number is in the declared unit); anything else is malformed;
conversion is host behavior outside grading; the tag is read as source text. (b) Define
compatible units and conversions in the spec.

## Evidence

- `docs/responses.md:133-142`: A response with a foreign unit is both malformed and scored 0

  ```markdown
  Implementations MAY do unit conversions and rewrite the response to a canonical
  form, but they MUST reject a response with a unit that is not compatible with
  the question's unit with grade zero.
  ```
- `examples/grading/numeric.yaml:1-20`: No grading vector covers units

  ```yaml
  #   examples: { response: number | string, score: 0 | 1 | "malformed" }[]
  ```
- `docs/references/grammar.md:89`: UNIT does not say if a markdown escape or code span in the
unit is source text

  ```lark
  UNIT         : /[^()\[\]\t\n\v\f\r
\x85\xa0\u1680\u2000-\u200a\u2028\u2029\u202f\u205f\u3000]+/
  ```

## Impact

- **A response with a foreign unit is both malformed and scored 0:** The same response gets a
score in one implementation and an error in another.
- **No grading vector covers units:** The unit rules of responses.md have no shared test, so
implementations can drift.
- **UNIT does not say if a markdown escape or code span in the unit is source text:** Two
implementations can store different `unit` strings for the same tag.

## Suggested fix

- **A response with a foreign unit is both malformed and scored 0:** Decide one rule: (a) a
unit other than the declared one is always malformed, and conversion is outside the spec; or
(b) define compatible units and the conversion, and malformed only for incompatible ones. State
 that a bare number is in the declared unit, and how units compare.
- **No grading vector covers units:** After the unit rule is decided, add the `{value, unit}`
form to the header and vectors for each case.
- **UNIT does not say if a markdown escape or code span in the unit is source text:** State
that the tag line, including the unit, is read as source text with no markdown inline
processing, or exclude `\` and the backtick from `UNIT`.

## Acceptance criteria

- [ ] One rule for a foreign unit
- [ ] A bare number for a question with a unit is defined
- [ ] The unit of a tag is defined as source text or escapes are excluded
- [ ] Vectors with the {value, unit} form

--- END ISSUE 3 ---
````

````
--- ISSUE 4 ---
# [Spec] Align the numeric diagnostics for domain and tolerance

**Labels:** medium, low

## Problem

A non-whole answer under integer gets two codes, under fraction only an info despite a SHOULD;
missing-tolerance fires on exact fractions; Additional Rules has rows the prose does not state
and SHOULD rules of the parser output have no row. Recommended: answer-outside-domain covers a
non-whole answer under integer and fraction (warning, 'should'); domain-mismatch covers only
the absolute tolerance; missing-tolerance only for decimal; the Tolerances section states the
non-negative and percentage rules; Answer representation says its rules apply to the parser
output and have no diagnostic.

## Evidence

- `docs/question-types/numeric.md:210, docs/question-types/numeric.md:214`: A non-whole answer
of an integer question gets two diagnostics

  ```markdown
  | answer             | warning  | answer-outside-domain          | must be a whole number
when `domain` is "integer"[^6]              |
  ```
- `docs/question-types/numeric.md:218, docs/question-types/numeric.md:229-230`: missing-
tolerance fires on fraction answers, where an exact match is meaningful

  ```markdown
  | tolerance          | info     | missing-tolerance              | is not defined for a
`decimal` or `fraction` answer[^5]            |
  ```
- `docs/question-types/numeric.md:111-113`: A non-whole answer under domain fraction is a
SHOULD, but only an info diagnostic

  ```markdown
  When `domain` is `fraction`, such a document SHOULD give `answer` as a
  fraction string (`"3/4"`) or as an integer (`2` or `"2"`). A number that is
  not whole, such as `0.75`, is a decimal and contradicts the domain.
  ```
- `docs/question-types/numeric.md:211-218`: Additional Rules has rows that the prose does not
state

  ```markdown
  | tolerance.absolute | error    | schema-error                   | must not be negative
(schema)                                      |
  ```
- `docs/question-types/numeric.md:90-91, docs/question-types/numeric.md:100-102`: SHOULD rules
of Answer representation are not in Additional Rules

  ```markdown
  An integer number like `42` or `-1` SHOULD be coerced to an integral value.
  It SHOULD be kept as a string if it is not representable in the 32-bit signed
  integer range.
  ```
- `docs/question-types/numeric.md:152-153`: The Tolerances section does not map a percentage to
 tolerance.relative

  ```markdown
  where tol is calculated as `|answer| × percentage_number ÷ 100`.
  ```

## Impact

- **A non-whole answer of an integer question gets two diagnostics:** Duplicate diagnostics, or
 implementations that report only one of them; corpus lint files cannot tell which is right.
- **missing-tolerance fires on fraction answers, where an exact match is meaningful:** Every
exact fraction question gets a lint that suggests a change that turns it into a decimal
question.
- **A non-whole answer under domain fraction is a SHOULD, but only an info diagnostic:** Two
cases with the same text ("contradicts the domain") get different severities, and the SHOULD
````

```
has no warning.
- **Additional Rules has rows that the prose does not state:** A reader of the prose does not
learn these rules; the table is the only source.
- **SHOULD rules of Answer representation are not in Additional Rules:** The requirement levels
 in the text and in the table do not agree.
- **The Tolerances section does not map a percentage to tolerance.relative:** An author who
writes YAML can put `5` for 5%, which accepts a response 500% off (only an info diagnostic
reports it).

## Suggested fix

- **A non-whole answer of an integer question gets two diagnostics:** Decide which code covers
the answer and which covers the tolerance (for example, `answer-outside-domain` for the answer,
 `domain-mismatch` only for a decimal absolute tolerance under `integer`/`fraction`), and use
"should" for a warning.
- **missing-tolerance fires on fraction answers, where an exact match is meaningful:** Restrict
 `missing-tolerance` to the `decimal` domain, or rewrite the footnote with a reason that holds
for fractions.
- **A non-whole answer under domain fraction is a SHOULD, but only an info diagnostic:** Decide
 one level for a non-whole answer under a narrower domain: extend `answer-outside-domain` to
`fraction`, or change the SHOULD to MAY.
- **Additional Rules has rows that the prose does not state:** Add the rules to the
"Tolerances" section: both tolerances are non-negative, a relative tolerance is a fraction
(`5%` is `0.05`), and a relative tolerance around a zero answer and a missing tolerance on a
decimal answer are flagged.
- **SHOULD rules of Answer representation are not in Additional Rules:** State that the rules
of "Answer representation" apply to the parser output and have no diagnostic, or reword them
without RFC 2119 keywords.
- **The Tolerances section does not map a percentage to tolerance.relative:** State that `+-
p%` is stored as `tolerance.relative: p / 100` and give the formula as `|answer - response| ≤
|answer| × relative`.

## Acceptance criteria

- [ ] Each defect has one code and one level
- [ ] Every Additional Rules row is stated in the prose
- [ ] Corpus lint files follow the new rows

--- END ISSUE 4 ---
```

````
--- ISSUE 5 ---
# [Spec] Decide whether numeric questions have feedback

**Labels:** medium

## Problem

Numeric has no feedback for an incorrect response. Options: (a) recommended: add
`incorrectFeedback`, as short-answer and fill-in blanks; (b) also add `reject` values with
tolerance and feedback for known wrong answers; (c) state that numeric has no feedback by
design.

## Evidence

- `docs/question-types/numeric.md:192-202`: Numeric questions have no feedback for responses

  ```markdown
  Grading is binary: a response inside the accepted tolerance scores 1, and any
  other response scores 0.
  ```

## Impact

Authors move numeric questions to short-answer regexes to give feedback, and lose tolerance and
 domain checks.

## Suggested fix

Either state that numeric has no feedback by design, or add `incorrectFeedback` (as in short-
answer and the blanks of fill-in) and, optionally, `reject` values with tolerance and feedback.

## Acceptance criteria

- [ ] numeric.md states the decision
- [ ] If added: schema, frontmatter table and vectors

--- END ISSUE 5 ---
````

````
--- ISSUE 6 ---
# [Spec] Numeric blanks of fill-in: indentation, domain and decimalPlaces

**Labels:** medium

## Problem

numeric_def has no leading ws?, and a Markdown numeric blank cannot set domain or
decimalPlaces. Recommended: add ws?; add frontmatter maps blank id -> domain and blank id ->
decimalPlaces, as for incorrectFeedback (or state the limit).

## Evidence

- `docs/question-types/fill-in.md:158, docs/question-types/fill-in.md:201`: numeric_def has no
leading ws?, unlike the numeric answer and the other blank definitions

  ```lark
  numeric_def      : "[^" SLUG "/numeric" unit? "]" ws? ":" ws? numeric_body
  ```
- `docs/question-types/fill-in.md:204-209`: A numeric blank in Markdown cannot declare domain
or decimalPlaces

  ```markdown
  The `numeric_body` and `unit` non-terminals represent, respectively, a numeric
  value with tolerance and an optional unit of measurement. Both are defined in
  the [body](numeric.md#body) section of the numeric question type. The `blank`
  ```

## Impact

- **numeric_def has no leading ws?, unlike the numeric answer and the other blank
definitions:** An indented numeric blank definition is a parse error by the grammar while the
same indentation is valid on the other definitions. Implementations can disagree on which
reading to follow.
- **A numeric blank in Markdown cannot declare domain or decimalPlaces:** An author cannot
write a decimal blank with an integer answer (`42` in a `decimal` domain), or set the precision
 of a blank, in Markdown. The YAML form can express questions that the Markdown form cannot.

## Suggested fix

- **numeric_def has no leading ws?, unlike the numeric answer and the other blank
definitions:** Add `ws?` at the start of `numeric_def` in both snippets, or remove it from all
definitions and state the indentation rule once.
- **A numeric blank in Markdown cannot declare domain or decimalPlaces:** Either state that a
Markdown numeric blank always uses the inferred domain and places, or add a frontmatter map
(blank id to `domain`/`decimalPlaces`), as fill-in already does for `preAccept` and
`incorrectFeedback`.

## Acceptance criteria

- [ ] numeric_def starts with ws?
- [ ] fill-in.md states how a Markdown blank sets domain and decimalPlaces

--- END ISSUE 6 ---
````

````
--- ISSUE 7 ---
# [Spec] Limit the indentation of ws? at the start of a block

**Labels:** medium

## Problem

grammar.md limits leading ws? to three spaces only inside list items. Recommended: state the
same limit (tabs expanded) for ws? at the start of any block, as CommonMark does.

## Evidence

- `docs/references/grammar.md:59-62`: ws? at the start of a block has no limit of three spaces

  ```
  When a rule starts with `ws?` at the start of a block, implementations MUST
  read the source text and accept only spaces and tabs there.
  ```

## Impact

An implementation that applies the grammar to the source text accepts an indented code block as
 the answer. An implementation that uses the CommonMark block structure rejects it. The same
document gives a question or a parse error.

## Suggested fix

State in grammar.md:59-62 that `ws?` at the start of a block stands for at most three spaces,
with tabs expanded as in grammar.md:70-71, the same as for a list item.

## Acceptance criteria

- [ ] grammar.md states the limit
- [ ] A corpus case with four spaces before a numeric tag

--- END ISSUE 7 ---
````

````
--- ISSUE 8 ---
# [Spec] Explain rejected number spellings

**Labels:** info

## Problem

U+2212, `±`, the decimal comma, digit group spaces and exponents are a plain parse-error.
Options: (a) recommended: list them in Body and add a lint hint for U+2212, `±` and the decimal
 comma; (b) accept some of them.

## Evidence

- `docs/question-types/numeric.md:68-74`: Common number spellings are a bare parse-error with
no lint to explain them

  ```lark
  numeric_body : ws? sign? value tolerances? ws?
  ```

## Impact

Authors who write Brazilian or typographic number forms get a generic parse-error.

## Suggested fix

List the rejected forms in the Body section, and consider a lint code with a hint for U+2212,
`±` and the decimal comma.

## Acceptance criteria

- [ ] Body lists the rejected forms
- [ ] If a hint is added, it is in lint-codes.md and the corpus

--- END ISSUE 8 ---
````

````
--- ISSUE 9 ---
# Mechanical fixes

**Labels:** mechanical, high, low

## Problem

Each fix below follows from its finding and needs no design decision, so they can be applied
together in one pass and reviewed by reading the diff. Severity still applies: a mechanical fix
 can matter a lot.

### F12 (high): The fraction rule accepts a zero denominator that the corpus rejects as parse-
error

`docs/question-types/numeric.md:78, docs/question-types/numeric.md:109-110`

```lark
fraction     : INTEGER "/" INTEGER
```

**Suggested fix:** Use a terminal with no zero for the denominator, for example `fraction :
INTEGER "/" DENOMINATOR` with `DENOMINATOR : /[1-9][0-9]*/`, as the schema pattern and the
corpus already do.

### F7 (low): Schema descriptions of decimalPlaces and relative disagree with the spec

`schema/numeric.yaml:48-50, schema/numeric.yaml:86-89`

```yaml
The number of decimal places used in the decimal value representation.
```

**Suggested fix:** Copy the definitions of numeric.md: "the precision of the comparison: the
response and the answer are rounded to this many places" and `|submitted - answer| <= |answer|
* relative`.

### F15 (low): numeric_def gives two ws? in a row, which makes the grammar ambiguous

`docs/question-types/fill-in.md:158, docs/question-types/fill-in.md:201`

```lark
numeric_def : "[^" SLUG "/" "numeric" unit? "]" ws? ":" ws? numeric_body
```

**Suggested fix:** Remove the `ws?` after `":"` in `numeric_def`, as `answer` does.

### F16 (low): The two numeric_def snippets of fill-in.md are written differently

`docs/question-types/fill-in.md:158, docs/question-types/fill-in.md:201`

```lark
numeric_def : "[^" SLUG "/" "numeric" unit? "]" ws? ":" ws? numeric_body
```

**Suggested fix:** Use the same text in both snippets, or keep the rule in one place and refer
to it.

### F24 (low): The intro says that a numeric question always declares a tolerance

`docs/question-types/numeric.md:5-6, docs/question-types/numeric.md:36-37`

```markdown
Numeric questions expect a single numeric answer. They declare both the correct
answer and tolerance.
```

**Suggested fix:** Write "the correct answer and, optionally, a tolerance" in both places.

### F29 (low): Schema description of domain omits inference and its effect
````

`schema/numeric.yaml:43`

```yaml
description: The domain/kind of number for the response.
```

Suggested fix: Describe `domain` as in numeric.md: inferred when absent (integer < fraction < decimal, the widest of the answer and the absolute tolerance), and it decides which responses are valid.

Acceptance criteria

- [ ] F12: The fraction rule accepts a zero denominator that the corpus rejects as parse-error
- [ ] F7: Schema descriptions of decimalPlaces and relative disagree with the spec
- [ ] F15: numeric_def gives two ws? in a row, which makes the grammar ambiguous
- [ ] F16: The two numeric_def snippets of fill-in.md are written differently
- [ ] F24: The intro says that a numeric question always declares a tolerance
- [ ] F29: Schema description of domain omits inference and its effect

--- END ISSUE 9 ---